A Unified Performance and Parallelism Model for LLM Inference

Modeling Memory, FLOPs, and Collectives for Efficient Transformer Inference at Scale

Author: Yue Lu
Date: November 2025

Keywords:
LLM inference, Transformer, parallelism, tensor parallelism, expert parallelism, sequence parallelism,
pipeline parallelism, distributed systems, KV cache, collective communication, latency, throughput,
cluster topology, performance modeling

Table of Contents

- 0. Introduction and Notation
  - 0.1 Symbol Notation
  - 0.2 Parallelism Architecture
- 1. Memory Footprint
  - 1.0 Parameter Definitions (P vs W)
  - 1.1 Model Parameter Memory
  - 1.2 Activation Memory (Per-token Working Memory)
  - 1.3 KV Cache Memory
  - 1.4 Per-device Memory Footprint After Parallelism Sharding
- 2. Memory Traffic During Decoding
  - 2.1 Model Parameter Traffic $T_{\theta,\text{device}}$
  - 2.2 Activation Traffic $T_{\text{act,device}}$
  - 2.3 KV Cache Traffic $T_{\text{KV,device}}$
  - 2.4 Total and Effective Traffic
  - 2.5 Static Memory Footprint vs. Memory Traffic
- 3. Compute (FLOPs) per Token
  - 3.1 Q/K/V and Output Projections
  - 3.2 Attention Scores and Value Application
  - 3.3 FFN FLOPs (Unified Dense + MoE)
  - 3.4 LayerNorm and Elementwise FLOPs
  - 3.5 Per-Device FLOPs per Layer Under TP, SP, EP, and PP
  - 3.6 Prefill FLOPs (Supplementary Reference)
- 4. Compute vs. Memory Bound (Roofline Model)
  - 4.1 Operational Intensity (Ops:Byte)
  - 4.2 Compute-Bound Time
  - 4.3 Memory-Bound Time
  - 4.4 Local Device Time Bound
- 5. Communication Time During Decoding
  - 5.1 Pipeline Parallel (PP) Hop
  - 5.2 Expert Parallel (EP) All-to-All (MoE Dispatch and Combine)
  - 5.3 Tensor Parallel (TP) Communication
  - 5.4 Sequence Parallel (SP) Communication
  - 5.5 Total Communication Time Per Token on a PP Stage

- 6. End-to-End Latency, Throughput, and Partition Strategy
  - 6.1 Model Partition Strategy from HBM Constraints
  - 6.2 Local and Networking Per-Token Latency
  - 6.3 TPS and TTPS — Pipeline Throughput
  - 6.4 TTFT — Time To First Token (Prefill on a Separate Cluster)

---

0. Introduction and Notation

This document presents a unified performance model for large-scale LLM inference on multi-GPU/NPU clusters. The focus is **autoregressive decoding**, where performance depends on:

- model dimensions $(H, n_q, n_{kv}, L, I_{\text{dense}}, I_{\text{moe}})$,
- parallelism configuration (DP, PP, EP, TP, SP),
- compute and memory efficiency (FlashAttention),
- KV-cache layout and size,
- interconnect topology and collective bandwidth.

0.1 Symbol Notation

Our symbol notation used in this documentation is aligned with:

- Megatron-LM (Shoeybi et al., 2019)
- DeepSpeed-MoE (Rajbhandari et al., 2022)
- FlashAttention (Dao et al., 2022, 2023)
- NVIDIA CUTLASS / cuBLAS GEMM dataflow
- Recent MoE and long-context LLM deployments

Parallelism Dimensions

- DP — Data Parallelism
  The number of full model replicas. Each DP replica handles disjoint input batches. DP is limited by total cluster size and the memory footprint of the sharded model:

$$DP = \left\lfloor \frac{N_{\text{GPUs}}}{PP \cdot EP \cdot TP \cdot SP} \right\rfloor$$

- PP — Pipeline Parallelism
  Layers split into pipeline stages. Let total layers be L and pipeline degree be PP, then each stage holds $L_{\text{stage}} = \frac{L}{PP}$

- TP — Tensor Parallelism
  Splits large matrix multiplies across devices. Used to shard attention Q/K/V/O projections and FFN/MoE matrices. Commonly “column” or “row” parallel as in Megatron-LM.

- EP — Expert Parallelism
  For MoE models: experts are partitioned across devices. Tokens are routed via all-to-all within an EP group, so each device hosts a subset of experts.

- SP — Sequence Parallelism
  In this document, SP means **ring-attention-style KV sharding** for inference: the

sequence dimension is partitioned across devices for KV storage, while each rank still computes attention for all queries via ring communication.

Model Dimensions

- L — Number of transformer layers.
- V — Vocabulary size (number of tokens in the tokenizer).
- H — Hidden size (model dimension). Hidden-state dimension H applies to embeddings, LM head, FFN blocks, and attention projections.
- n_q — Number of query heads.
- d_{head} — Head dimension, typically H/n_q.
- n_{kv} — Number of KV heads (in GQA, $n_{kv} < n_q$).
- $H_{kv} = n_{kv} d_{\text{head}}$ — Total KV projection dimension. KV dimension H_{kv} applies specifically to K/V projections and cache storage.
- I — Unified FFN intermediate dimension
  - $I = I_{\text{dense}}$ for dense layers
  - $I = I_{\text{moe}}$ for MoE layers
- I_{eff} — Unified FFN intermediate dimension for FLOPs calculation
  - $I_{\text{eff}} = I_{\text{dense}}$ for dense layers
  - $I_{\text{eff}} = kI_{\text{moe}}$ for MoE layers
- N_{exp} — Number of experts per MoE layer
- N_{eff} — Unified number of experts per MoE layer for FLOPs calculation
  - $N_{\text{eff}} = 0$ for dense layers
  - $N_{\text{eff}} = N_{\text{exp}}$ for MoE layers
- k — Number of experts selected per token (top-k routing in MoE)

Sequence / Batch and Precision

- S — Sequence length (tokens in context)
- B — Batch size (number of sequences)
  For decoding, B does *not* affect memory footprint or per-token FLOPs. It only appears in prefill (Section 3.6 and Section 6.4).
  Thus, throughout this document, B is included only where prefill behavior is analyzed.
- b — Bytes per element (e.g., bf16 ⇒ $b = 2$, fp8 ⇒ $b = 1$)

Static and Dynamic Memory

Parameter size

- P_{attn} — Attention parameter size
- P_{FFN} — Unified FFN/MoE parameter size
- P_{emb} — Embedding parameter size
- P_{lm} — LM head (0 if tied) parameter size

Memory capacity quantities (how many bytes are **stored** in HBM)

- $M_{\theta,\text{device}}$ — Parameter memory on this device
- $M_{\text{KV,device}}$ — KV cache storage for keys + values
- $M_{\text{act,device}}$ — Activation working memory on this device (per token during decoding)
- M_{HBM} — Available HBM capacity per device

Memory traffic quantities (how many bytes **move** between HBM and compute per token)

- $T_{\theta,\text{device}}$ — Parameter traffic (weights read per token)
- $T_{\text{KV,device}}$ — KV traffic (KV read + write per new token)
- $T_{\text{act,device}}$ — Activation traffic (intermediate reads/writes)
- $T_{\text{token,device}}$ — Total per-token traffic on this device
- $T^{eff}_{\text{token,device}}$ — Effective traffic after FlashAttention-style optimizations

We also define:

- c_{act} — Empirical number of unavoidable hidden-state I/O per layer that remains even after FlashAttention-style optimizations.

Device Compute and Bandwidth

- N_{GPUs} — Total number of GPUs/devices available in the cluster
- R_{GPU} — Effective compute throughput of the device (FLOPs/s)
- $B_{\text{eff,mem}}$ — Effective HBM bandwidth (bytes/s)

Networking

- $\alpha_{TP}, \alpha_{EP}, \alpha_{SP}, \alpha_{PP}$ — Startup latency constants for TP, EP, SP, and PP collectives, respectively (modeled in an α–B style latency + bandwidth formulation)
- $B_{\text{eff,TP}}$, $B_{\text{eff,EP}}$, $B_{\text{eff,SP}}$, $B_{\text{eff,PP}}$ — Effective interconnect bandwidths for TP, EP, SP, and PP communication
- n_{TP} — Number of TP collective iterations per-layer per-token
- n_{EP} — Number of EP collective iterations per-layer per-token
- n_{SP} — Number of SP collective iterations per-layer per-token (typically 1, during Attention)

Model FLOPs and Compute Quantities

FLOPs (floating point operations) per token and per layer:

- F_Q, F_K, F_V, F_O — FLOPs for the Q, K, V, and output projections
- F_{proj} — Combined Q/K/V/O projection FLOPs
- F_{score} — FLOPs to compute attention scores ($QK^\top$)
- F_{value} — FLOPs to apply attention weights to values
- $F_{\text{attn,KV}}$ — FLOPs for the attention score + value application terms
- F_{attn} — Total attention FLOPs per layer (projections + attention score/value application)

FFN and MoE FLOPs:

- $F_{\text{ffn,dense}}$ — Dense FFN FLOPs per layer
- F_{router} — Router FLOPs per token for MoE
- F_{expert} — FLOPs per expert MLP (for one expert and one token)
- $F_{\text{ffn,moe}}$ — MoE FFN FLOPs per layer
- F_{ffn} — Unified FFN FLOPs (dense or MoE), used throughout Section 5

Layer and token FLOPs:

- F_{norm} — FLOPs for normalization (LayerNorm/RMSNorm), typically $c_{\text{norm}}H$.
- c_{norm} — Constant capturing the norm FLOP coefficient (e.g., 5–10).
- F_{layer} — Total FLOPs per layer (attention + FFN + norm).
- $F_{\text{layer,device}}$ — FLOPs per layer per device after TP/EP sharding.
- $F_{\text{token,device}}$ — Total FLOPs per generated token on this device (decoding).
- F_{prefill} — Total FLOPs for prefill (processing the full input sequence before decoding).

Time, Throughput, and Roofline Quantities

Local (on-device) & communication timing:

- t_{compute} — Per-token compute time, $F_{\text{token,device}}/R_{\text{GPU}}$.
- t_{mem} — Per-token memory time, $T^{eff}_{\text{token,device}}/B_{\text{eff,mem}}$.
- t_{local} — Local per-token roofline time: $\max(t_{\text{compute}}, t_{\text{mem}})$.
- t_{TP}, t_{EP}, t_{SP}, t_{PP} — Communication time per token for TP/EP/SP/PP.
- t_{comm} — Combined communication work per token.
- $t_{\text{prefill,local}}$ — Prefill compute + memory time.
- $t_{\text{prefill,comm}}$ — Prefill communication time.
- t_{startup} — Kernel warmup and graph-capture overhead.
- t_{token} — Effective per-token time, accounting for compute, memory, and communication.
- ρ — Overlap factor $\in [0, 1]$ representing the fraction of local compute/memory time utilized to hide communication. Used in the unified latency model:

$$t_{\text{token}} = t_{\text{local}} + \max(0, t_{\text{comm}} - \rho \cdot t_{\text{local}})$$

Throughput & end-to-end latency:

- TPS_{single} — Per-DP replica throughput: token per second

- $TTPS$ — Global throughput across DP replicas: total token per second TTPS is the correct metric for end-to-end system throughput when multiple DP replicas serve independent requests.
- $TTFT$ — Time to first token: the amount of time an LLM takes to generate the first token in its response after receiving an input or prompt.

0.2 Parallelism Architecture

We adopt a fixed nesting order for all parallelism dimensions:

$$\boxed{\mathrm{DP} \;\rightarrow\; \mathrm{PP} \;\rightarrow\; \mathrm{EP} \;\rightarrow\; \mathrm{TP} \;\rightarrow\; \mathrm{SP}}$$

This order reflects how model state is partitioned and reused during inference. Each level depends on all outer levels having already determined the weight placement, token routing, or tensor partitioning.

Summary of Nesting Rationale

Level	What it partitions	Why this ordering is required
DP	Entire model replica	Must wrap all state; inner groups cannot cross DP boundaries.
PP	Layers	Layer ownership must be decided before experts/tensor shards are assigned.
EP	Experts	Expert placement must be fixed before tensor sharding splits expert matrices.
TP	Weight matrices	TP defines weight shards used identically across all SP ranks.
SP	KV cache sequence dimension	KV is activation state only; must be sharded after all weight placement.

DP is the outermost level (replicated model weights)

Why:

- DP creates fully independent model replicas for throughput.
- No weight partitioning happens inside DP groups.
- All deeper parallelism dimensions (PP, EP, TP, SP) apply **within** each DP replica.

Therefore:

$$\mathrm{DP\ is\ always\ outermost}$$

PP is inside DP (layers assigned before experts/tensor sharding)

PP splits *layers* of the model across devices.

Why PP must come before EP, TP, SP:

- PP determines **which layers live on which devices**.
- Only after PP is fixed can we partition:
  - Experts (EP) across devices
  - Tensor dimensions (TP) inside each block
  - KV sequence partitions (SP) inside each attention layer
- PP stages own their local KV cache and local weights.

Therefore:

$$\mathrm{DP} \to \mathrm{PP}$$

EP is inside PP (expert groups belong to specific layers)

EP distributes MoE experts **within the layers assigned by PP**.

Why EP must come before TP and SP:

- Expert weights must be assigned to EP ranks resolved **before** tensor-parallel shards apply.
- TP cannot shard expert weights until expert placement is set by EP.
- SP does not affect expert routing; EP must be outer.

In addition, for each MoE layer with N_{exp} experts, the expert-parallel degree must satisfy

$$EP \leq N_{\text{exp}},$$

so that each EP rank can host at least one expert. In practice, EP is usually chosen to divide N_{exp} (e.g., $EP \mid N_{\text{exp}}$), but the only hard requirement in this model is $EP \leq N_{\text{exp}}$.

Therefore:

$$\mathrm{DP} \to \mathrm{PP} \to \mathrm{EP}$$

TP is inside EP (tensor sharding within a defined expert/layer partition)

TP splits matrices *within each expert or dense block*.

Why TP must come before SP:

- TP shards Q/K/V/O projections, MLP layers, and expert MLPs.
- After TP, each rank holds a **fraction of H or H_{kv}**.
- SP requires all SP ranks to share identical TP-sharded weights.

Therefore:

$$\mathrm{DP} \to \mathrm{PP} \to \mathrm{EP} \to \mathrm{TP}$$

SP is innermost (KV sharding after all weights are fixed)

SP shards the **KV cache**, not model parameters.

Why SP must be last:

- SP only partitions activation state (KV), not weights.
- SP requires all TP ranks to already have consistent weight shards.
- Only after DP/PP/EP/TP are fixed can we shard the KV cache.

Therefore:

$$\mathrm{DP} \to \mathrm{PP} \to \mathrm{EP} \to \mathrm{TP} \to \mathrm{SP}$$

---

1. Memory Footprint

This section defines parameter sizes and memory footprint for a given set of model parameters. The memory footprint include those from model weights, per-token activation/working memories, and KV cache. We avoid model-wide parameter aggregation here and instead focus on **per-layer** quantities, because pipeline-parallel stages own disjoint sets of layers.

All parameter definitions assume stored precision of b bytes per element (e.g., bf16 = 2 bytes).

1.0 Parameter Definitions (P vs W)

For any weight matrix W, the parameter count is

$$P(W) = \text{number of elements in } W$$

Total stored parameter memory is

$$M_\theta = P \cdot b$$

1.1 Model Parameter Memory

Embedding and LM Head Parameters

Modern LLM architectures (GPT-3/4, LLaMA families, PaLM, Qwen, DeepSeek, etc.) typically use embedding dimension $E = H$, so all internal projections operate on vectors in $\mathbb{R}^H$.

For the token embedding:

$$W_{\text{emb}} \in \mathbb{R}^{V \times H}, \quad P_{\text{emb}} = VH$$

If the LM head is tied with embeddings, i.e. $W_{lm} = W_{emb}^T$ so that W_{emb} can be re-used:

$$P_{\text{lm}} = 0$$

If untied:

$$W_{\text{lm}} \in \mathbb{R}^{H \times V}, \quad P_{\text{lm}} = VH$$

Attention Parameters

For hidden size H, head dimension d_{head}, and KV dimension $H_{kv} = n_{kv}\, d_{\text{head}}$:

- $W_Q \in \mathbb{R}^{H \times H}$
- $W_K \in \mathbb{R}^{H \times H_{kv}}$
- $W_V \in \mathbb{R}^{H \times H_{kv}}$

- $W_O \in \mathbb{R}^{H_{kv} \times H}$

Parameter counts:

$$P_Q = H^2, \qquad P_K = HH_{kv}, \qquad P_V = HH_{kv}, \qquad P_O = H_{kv}H$$

We define attention parameters:

$$P_{\text{attn}} = P_Q + P_K + P_V + P_O$$

Unified FFN Parameters (Dense or MoE)

Each transformer layer contains an FFN module. In modern LLM architectures, this FFN is almost always implemented as a gated MLP (GeLU/SiLU/GLU/SwiGLU–style) with **three** linear projections:

- a gate projection,
- an "up" projection (expansion),
- a "down" projection (contraction).

For a hidden size H and FFN intermediate dimension I, this yields an FFN parameter count

$$P_{\text{FFN}} = 3HIN_{\text{exp}}.$$

Here N_{exp} denotes the number of experts **per layer**:

- **For a dense MLP model:** $I = I_{\text{dense}},\ N_{\text{exp}} = 1.$
- **For a MoE model:** $I = I_{\text{moe}}$, with $N_{\text{exp}} > 1.$

These two model cases are mutually exclusive **per layer**.

LayerNorm parameters

LayerNorm or RMSNorm contain $\mathcal{O}(H)$ parameters (scale and optional bias). These are negligible compared to attention and FFN weights and are omitted in scaling formulas.

1.2 Activation Memory (Per-token Working Memory)

During autoregressive decoding, the model processes **one token at a time**. As a result, the only activations that need to be stored in memory are the **temporary, layer-local working buffers** used in the forward pass of the *current token*.

These activations are **not reused** across layers or across tokens and therefore are:

- **not dependent on sequence length** S,
- **not dependent on batch size** B,
- **not EP- or TP-sharded**,
- and **extremely small** relative to model parameter memory (Section 1.1) and KV cache memory (Section 1.3).

Below we account for all activation tensors in a model layer that must be alive **concurrently** for one decoding token.

Q, K, V projections

For the current hidden state $h \in \mathbb{R}^H$, the layer computes:

- $Q \in \mathbb{R}^H$
- $K \in \mathbb{R}^{H_{kv}}$
- $V \in \mathbb{R}^{H_{kv}}$

This contributes:

$$H + 2H_{kv}$$

Attention score accumulation buffer (FlashAttention-like kernels)

Attention score computation normally requires a temporary buffer.
FlashAttention-style fused kernels avoid storing full S-length score vectors and instead use a **single internal workspace** of size H during streaming softmax.

This adds:

$$+H$$

Attention output buffer

After applying attention weights to V and combining across heads, we form the attention output $O_{\text{attn}} \in \mathbb{R}^H$.

This output must exist before the output projection is applied, contributing:

$$+H$$

FFN working buffer

Following attention and normalization, the FFN block needs at least one temporary buffer of size H to hold either the FFN input or the FFN output before residual addition.
Even with kernel fusion, this buffer cannot always overlap with the attention intermediates.

This adds:

$$+H$$

Summing all simultaneously required buffers:

$$P_{\text{act}} = 4H + 2H_{kv}$$

In bytes:

$$M_{\text{act,layer}} = (4H + 2H_{kv}) \cdot b$$

This footprint is **small** compared to parameter memory and KV cache, which is why activation memory does not limit decoding capacity under typical inference workloads.

1.3 KV Cache Memory

Section 1.1 described the memory footprint of model parameters (static), and Section 1.2 covered the activation memory required during decoding (per-token, dynamic). This section describes the **KV cache**, which is a *runtime* structure generated during the **pre-fill phase**, when the model processes the entire input sequence of length S.

During pre-fill, each attention layer produces:

- one key vector of dimension H_{kv},
- one value vector of dimension H_{kv},

for each input token. Because decoding adds only one new token at a time, the vast majority of KV memory comes from **pre-fill**, not decoding.

For a single attention layer, the KV cache consists of:

- Keys: $K \in \mathbb{R}^{S \times H_{kv}}$
- Values: $V \in \mathbb{R}^{S \times H_{kv}}$

Thus, the total KV elements for one layer are:

$$P_{KV,layer} = S \cdot (2H_{kv}) = 2SH_{kv}$$

In bytes:

$$M_{\text{KV,layer}} = 2SH_{kv} \cdot b$$

This is **static** once pre-fill is complete; decoding contributes only an additional $2H_{kv}b$ per generated token, which is negligible relative to the full cache.

1.4 Per-device Memory Footprint After Parallelism Sharding

So far we've completed the all the memory footprint estimation for a model layer. When we introduce different parallelism schemes, some of these memories would be sharded by one or more of these parallelism dimensions, resulting in a somewhat complicated memory aggregateion per device. We now describe how these parameters are distributed across devices under PP/EP/TP/SP, and then derive simple modeling approximations.

Per-device Parameter Memory (TP, EP, PP)

Each transformer layer has two parameter groups:

- **Attention parameters** P_{attn}, that are sharded by **tensor parallelism (TP)** only.

- **FFN parameters** P_{FFN} are sharded by **TP and EP**. Dense FFN layers are treated as a special case with $EP = 1$, so the same formula applies to both dense and MoE FFNs.

For a *single layer* on a device, the stored parameter memory is

$$M_{\theta,\text{layer}} = \frac{P_{\text{attn}}\, b}{TP} + \frac{P_{\text{FFN}}\, b}{TP \cdot EP}$$

Pipeline parallelism (PP) assigns **disjoint sets of layers** to different stages.
Let L_s be the set of layers that live on PP stage s, and let $M_{\theta,\text{layer},\ell}$ be the per-layer memory from the expression above.

Excluding embeddings and LM head, the parameter memory per device on PP stage s is

$$M_{\theta,\text{layers}}^{(s)} = \sum_{\ell \in L_s} M_{\theta,\text{layer},\ell}$$

Embeddings and LM head appear only on two stages:

- **Intermediate PP stages** (no embedding / LM head):

$$M_{\theta,\text{device}}^{(\text{mid})} = M_{\theta,\text{layers}}^{(\text{mid})}$$

- **First PP stage** (with token embedding):

$$M_{\theta,\text{device}}^{(1)} = M_{\theta,\text{layers}}^{(\text{mid})} + \frac{P_{\text{emb}}\, b}{TP}$$

- **Final PP stage** (with LM head):

$$M_{\theta,\text{device}}^{(\text{PP})} = M_{\theta,\text{layers}}^{(\text{mid})} + \frac{P_{\text{lm}}\, b}{TP}$$

If each intermediate PP stage holds approximately L/PP layers of similar size, and we use representative per-layer values P_{attn} and P_{FFN}, then

$$M_{\theta,\text{device}}^{(\text{mid})} = \frac{L}{PP} M_{\theta,\text{layer}} = \frac{L}{PP} \left(\frac{H^2 + 3HH_{kv}}{TP} + \frac{3HIN_{\text{exp}}}{TP \cdot EP} \right) b$$

For capacity planning we use a worst-case PP stage budget, adding one $\frac{VH}{TP} b$ term to account for embedding/LM weights residing on boundary stages. Intermediate stages are slightly smaller. Therefore:

$$M_{\theta,\text{device}} = \frac{L}{PP} \left(\frac{H^2 + 3HH_{kv}}{TP} + \frac{3HIN_{\text{exp}}}{TP \cdot EP} \right) b + \frac{VH}{TP} b$$

For a **dense MLP model**: $I = I_{\text{dense}}$, and $N_{\text{exp}} = EP = 1$.

For a **MoE model**: $I = I_{\text{moe}}$, with $N_{\text{exp}} > 1$ and $EP \geq 1$.

Per-device Activation Memory

The per-layer working activation footprint for one decoding token is: $4H + 2H_{kv}$.

This footprint must exist for every layer on the PP stage. Since PP assigns whole layers to devices, each device hosts exactly L/PP layers. Therefore, the per-device activation memory during decoding is:

$$M_{\text{act,device}} = \frac{L}{PP}\,(4H + 2H_{kv})\,b$$

Per-device KV Cache Memory

Only attention layers produce KV cache.
The KV cache is sharded only across:

- **TP** — splits the channel dimension H_{kv},
- **SP** — splits the sequence dimension S.

EP and DP do not modify KV layout; PP only affects how many layers are assigned to a stage.

Therefore, the **per-device** KV memory footprint is:

$$M_{\text{KV,device}} = \frac{L}{PP}\frac{M_{\text{KV,layer}}}{TP \cdot SP} = \frac{L}{PP}\frac{(2SH_{kv})b}{TP \cdot SP}$$

which:

- For long-context inference (e.g., $S \in [16\text{k}, 128\text{k}]$), $S(2H_{kv})b$ is large enough that KV can exceed parameter memory unless aggressively reduced through TP and SP.
- Each decoded token adds only $2H_{kv}b$, which is negligible compared to the pre-fill KV footprint.

Total Per-device Static Memory Footprint

Summing all the memory footprint we dervive from section 1.1 - 1.4 together, we can therefore get the "minimum required" memory size for the device to host the model under a particular PP/EP/TP/SP partition.

$$M_{\text{device}}^{\text{total}} = M_{\theta,\text{device}} + M_{\text{act,device}} + M_{\text{KV,device}}$$

$$\boxed{M_{\text{device}}^{\text{total}} = \frac{L}{PP}\left[\frac{H^2 + 3HH_{kv}}{TP} + \frac{3HIN_{\text{exp}}}{TP \cdot EP} + (4H + 2H_{kv}) + \frac{2SH_{kv}}{TP \cdot SP}\right]b + \frac{VH}{TP}b}$$

For a dense model: $I = I_{\text{dense}},\ N_{\text{exp}} = EP = 1$

And for a MoE model: $I = I_{\text{moe}}$

---

2. Memory Traffic During Decoding

Section 1 quantified the *static* memory footprint of the model — how many bytes of parameters, KV cache, and activations must **fit** in device HBM.

This section instead focuses on **memory traffic per generated token**, i.e., the bytes that must flow between HBM and compute cores *during decoding*. This traffic directly determines the memory-bound component of decoding performance (Section 4's roofline model).

Crucial Distinction for Decoding: In autoregressive decoding (batch size ≈ 1), tokens are generated sequentially. Unlike prefill or training, where weights can be loaded once and reused across a large batch of tokens in SRAM, decoding requires the **entire model weight matrix** to be loaded from HBM for **every single generated token**.

Therefore, optimizations like FlashAttention or Fused-MLP do **not** reduce weight traffic; they only reduce the traffic of intermediate activations.

2.1 Model Parameter Traffic $T_{\theta,\text{device}}$

Following Section 1, we use:

- P_{attn}: Q/K/V/O projection parameters
- P_{FFN}: dense FFN (or non-expert MoE core) parameters

We drop P_{emb} and P_{lm} when modeling steady-state decoding, assuming they reside on boundary PP stages and do not bottleneck the central pipeline.

Attention parameter traffic

Because P_{attn} is defined **per layer**, a PP stage with $L_{\text{stage}} = L/PP$ layers has $L_{\text{stage}} P_{\text{attn}}$ attention parameters. These are sharded across TP devices.

Since every weight must be read per token:

$$T_{\theta,\text{attn}} = \frac{L}{PP} \cdot \frac{P_{\text{attn}}\, b}{TP}$$

FFN parameter traffic

Similarly, the FFN parameters P_{FFN} are sharded by both **TP** and **EP**. Although fused kernels (e.g., FlashMLP) avoid writing intermediate activations (like the gate tensor) to HBM, they still require reading the gate, up, and down projection weights fully.

$$T_{\theta,\text{FFN}} = \frac{L}{PP}\, \frac{P_{\text{FFN}}}{TP \cdot EP}\, b$$

Final parameter-traffic expression

Combining these terms:

$$T_{\theta,\text{device}} \approx \frac{L}{PP}\left(\frac{P_{\text{attn}}}{TP} + \frac{P_{\text{FFN}}}{TP \cdot EP}\right) b = \frac{L}{PP}\left(\frac{H^2 + 3HH_{kv}}{TP} + \frac{3HIN_{\text{exp}}}{TP \cdot EP}\right) b$$

For a dense MLP model: $I = I_{\text{dense}},\ N_{\text{exp}} = EP = 1$

And for a MoE model: $I = I_{\text{moe}}$

2.2 Activation Traffic $T_{\text{act,device}}$

Section 1 showed that the per-layer activation footprint for a single decoding token is small. However, without optimization, the traffic to read/write these activations—especially the $S \times S$ attention scores—would be massive ($O(S^2)$).

The Role of FlashAttention

FlashAttention avoids this $O(S^2)$ traffic by tiling the attention computation in SRAM. It ensures that the large attention score matrix is never written to or read from global memory.

Because FlashAttention eliminates the dominant score traffic, the only activation traffic remaining is the **linear** $O(H)$ traffic for input/output of the hidden states and FFN buffers.

Empirical Activation Constant (c_{act})

We model this "residual" traffic as a small multiple of the hidden size:

$$T_{\text{act,layer}} \approx c_{\text{act}}\, H\, b$$

Empirical kernel traces from FlashAttention-2/3, xFormers, and TensorRT-LLM consistently show:

$$c_{\text{act}} \approx 8\text{–}12.$$

This constant accounts for unavoidable loads/stores of the hidden state, attention output, and FFN residuals that remain even after fusion.

Final activation-traffic expression

For a PP stage containing L/PP layers:

$$T_{\text{act,device}} = \frac{L}{PP}\, c_{\text{act}}\, H\, b$$

As with activation *memory*, this traffic is **not sharded** by TP or EP because fused kernels typically operate on the full hidden state (or rank-local equivalents) before sharding logic applies.

2.3 KV Cache Traffic $T_{\text{KV,device}}$

KV cache must be **fully read** for each new token to compute attention against the history. For large S, the write term (appending the new token) is negligible compared to reading the history.

The per-layer KV access is approximately:

$$T_{\text{KV,layer}} \approx 2SH_{kv}\, b$$

Consistent with Section 1, KV is sharded by **TP** (channel/head dimension) and **SP** (sequence dimension). Thus each device only sees a $\frac{1}{TP \cdot SP}$ shard of this per-layer traffic.

For a PP stage with L/PP layers, the **per-device KV traffic per token** is:

$$T_{\text{KV,device}} \approx \frac{L}{PP} \cdot \frac{2SH_{kv}\, b}{TP \cdot SP}$$

FlashAttention does **not** reduce this term: keys and values from history must always be loaded to compute the current token's attention, regardless of tiling strategy.

2.4 Total and Effective Traffic

Combining the expressions derived in Sections 2.1–2.3, the **effective** total per-token traffic is:

$$T^{eff}_{\text{token,device}} \approx T_{\theta,\text{device}} + T_{\text{act,device}} + T_{\text{KV,device}}.$$

Substituting the corrected components yields the **final fully expanded expression**:

$$\boxed{T^{eff}_{\text{token,device}} \approx \frac{L}{PP}\left[\underbrace{\left(\frac{H^2 + 3HH_{kv}}{TP} + \frac{3HIN_{\text{exp}}}{TP \cdot EP}\right) b}_{\text{Weights (Must load 100\%)}} + \underbrace{c_{\text{act}}\, H\, b}_{\text{Activations}} + \underbrace{\frac{2SH_{kv}}{TP \cdot SP} b}_{\text{KV Cache}}\right]}$$

2.5 Static Memory Footprint vs. Memory Traffic (Important Distinction)

Sections 1 and 2 play different roles in the overall performance model:

Static Memory Footprint (Section 1) Determines whether a (DP, PP, EP, TP, SP) configuration can *fit* on a device (Capacity Constraint):

$$M^{\text{total}}_{\text{device}} \leq M_{\text{HBM}}$$

Memory Traffic (Section 2) Determines the *bandwidth-limited latency* per decoded token (Bandwidth Constraint):

$$t_{\text{mem}} = \frac{T^{eff}_{\text{token,device}}}{B_{\text{eff,mem}}}$$

This distinction is critical: Section 1 tells us **which parallelism configurations are viable**, while Section 2 tells us **how fast decoding can proceed** for those viable configurations.

---

3. Compute (FLOPs) per Token

During inference, the FLOPs required to generate a token depend on the transformer layer structure. We distinguish between:

- **Prefill FLOPs (GEMM-dominant):** $O(S^2)$ across the full input sequence
- **Decoding FLOPs (GEMV-dominant):** $O(S)$ for one additional token

This section focuses on **decoding FLOPs**, which determine TPS throughput. All FLOPs below represent **per-token**, **per-layer**, **decoding** FLOPs.

3.1 Q/K/V and Output Projections

Q, K, and V projections are vector–matrix multiplications of shapes:

- Q: $[1 \times H] \cdot [H \times H]$
- K: $[1 \times H] \cdot [H \times H_{kv}]$
- V: $[1 \times H] \cdot [H \times H_{kv}]$
- Output: $[1 \times H_{kv}] \cdot [H_{kv} \times H]$

Projection FLOPs

$$F_Q = 2H^2, \qquad F_K = 2HH_{kv}, \qquad F_V = 2HH_{kv}, \qquad F_O = 2HH_{kv}.$$

Where the factor 2 accounts for each multiply-accumulate pair in the standard GEMV convention.

Total

$$F_{\text{proj}} = 2H^2 + 6HH_{kv}$$

If $H_{kv} = H$, this reduces to $8H^2$.

3.2 Attention Scores and Value Application

During decoding, the newly generated token attends to all S cached tokens in the KV cache for this layer. Conceptually, for each layer we can treat the cached keys and values as:

- $K_{\text{cache}} \in \mathbb{R}^{S \times H_{kv}}$
- $V_{\text{cache}} \in \mathbb{R}^{S \times H_{kv}}$,

where $H_{kv} = n_{kv} d_{\text{head}}$ is the total KV projection dimension.

Scores (Q · K^T)

For the new token, attention scores are computed by taking a dot product between its query representation and each of the S cached key vectors of length H_{kv}(which matches standard

multi-head and GQA scaling up to small constant factors). In aggregate, this behaves like a streamed matrix–vector operation over an $S \times H_{kv}$ key matrix, giving:

$$F_{\text{score}} = 2SH_{kv}$$

The factor 2 reflects the standard GEMV convention (each multiply–accumulate counts as 2 FLOPs).

Value application (Attn · V)

After applying softmax to the scores, the attention weights are used to compute a weighted sum over the cached values. This again involves an effective $S \times H_{kv}$ matrix–vector style operation:

$$F_{\text{value}} = 2SH_{kv}$$

Total KV attention FLOPs

Combining the two:

$$F_{\text{attn,KV}} = F_{\text{score}} + F_{\text{value}} = 4SH_{kv}$$

This term captures the **sequence-length-dependent** attention cost during decoding.

3.3 FFN FLOPs (Unified Dense + MoE)

To match the parameter definitions in Section , we express FFN FLOPs using a **unified formulation** that works for both dense FFN layers and MoE layers.

Dense FFN FLOPs

A dense FFN consists of:

- an expansion GEMV: $H \to I$, and
- a contraction GEMV: $I \to H$.

Each GEMV costs $2HD$ FLOPs, giving:

$$F_{\text{ffn,dense}} = 4HI_{dense}$$

Dense FFN layers always have $EP = 1$.

MoE FFN FLOPs

For MoE layers:

- the **router** is applied to the full hidden vector:

  $$F_{\text{router}} = 2HN_{\text{exp}}$$

  where N_{exp} is the total number of experts.

- for each of the k selected experts for this token, the FFN computation is:

$$F_{\text{expert}} = 4HI_{\text{moe}}$$

Thus the MoE FFN FLOPs for one token in one layer are:

$$F_{\text{ffn,moe}} = F_{\text{router}} + k \cdot F_{\text{expert}} = 2HN_{\text{exp}} + k(4HI_{\text{moe}})$$

Unified FFN FLOP Term

We now define **effective FFN FLOP parameters**:

- Effective FFN "matrix" dimension:

$$I_{\text{eff}} = \begin{cases} I_{\text{dense}}, & \text{dense layer}, \\ kI_{\text{moe}}, & \text{MoE layer}, \end{cases}$$

- Effective router multiplicity:

$$N_{\text{eff}} = \begin{cases} 0, & \text{dense layer}, \\ N_{\text{exp}}, & \text{MoE layer}. \end{cases}$$

With these definitions, both dense and MoE FFN FLOPs can be written in a **single unified form**:

$$F_{\text{ffn}} = 4HI_{\text{eff}} + 2HN_{\text{eff}}$$

This matches:

- Dense layer: $F_{\text{ffn}} = 4HI_{\text{dense}} + 2H \cdot 0 = 4HI_{\text{dense}}$
- MoE layer: $F_{\text{ffn}} = 4H(kI_{\text{moe}}) + 2HN_{\text{exp}} = 4HI_{\text{moe}}k + 2HN_{\text{exp}}$

3.4 LayerNorm and Elementwise FLOPs

LayerNorm, RMSNorm, residual additions, and small elementwise operations all scale linearly with H. Although these costs are much smaller than attention and FFN FLOPs, we include them for completeness.

We write:

$$F_{\text{norm}} = c_{\text{norm}}H$$

where c_{norm} is a small implementation-dependent constant (typically 5–20).

In practice, F_{norm} is much smaller than the projection, attention, and FFN FLOPs. In the per-device FLOP expressions below, we will neglect F_{norm} and write approximate equalities ($\approx$) to avoid clutter, noting that this introduces only a small relative error.

3.5 Per-Device FLOPs per Layer Under TP, SP, EP, and PP

For a single decoding token, the FLOPs for one transformer layer are:

$$F_{\text{layer}} = F_{\text{proj}} + F_{\text{attn,KV}} + F_{\text{ffn}} + F_{\text{norm}},$$

where:

- $F_{\text{proj}} = 2H^2 + 6HH_{kv}$ (Section 3.1),
- $F_{\text{attn,KV}} = 4SH_{kv}$ (Section 3.2),
- F_{ffn} is dense or MoE (Section 3.3),
- $F_{\text{norm}} = c_{\text{norm}}H$ (Section 3.4).

To find **per-device FLOPs**, we apply sharding from TP, SP, EP and then multiply by the number of layers on the PP stage.

TP (Tensor Parallelism)

TP shards all **GEMV/GEMM-like** FLOPs:

- projections (F_{proj}),
- attention score/value FLOPs ($F_{\text{attn,KV}}$),
- FFN GEMMs (dense or MoE experts).

Thus:

$$F_{\text{tensor}}^{\text{device}} = \frac{1}{TP} F_{\text{tensor}}$$

Normalization FLOPs F_{norm} are tiny and unsharded:

$$F_{\text{norm}}^{\text{device}} = F_{\text{norm}}$$

SP (Sequence Parallelism)

SP shards the **sequence dimension** across SP ranks.
Thus **only** the sequence-dependent FLOPs:

$$F_{\text{attn,KV}} = 4SH_{kv}$$

are reduced:

$$F_{\text{attn,KV}}^{\text{device}} = \frac{1}{TP \cdot SP}(4SH_{kv})$$

SP does **not** reduce:

- F_{proj} (single token),
- F_{ffn} (dense or MoE),
- router FLOPs,
- normalization FLOPs.

EP (Expert Parallelism)

EP applies only to MoE layers:

- Router FLOPs use the full hidden state and are **not sharded**.
- Expert FFN GEMMs are sharded across EP:

$$F_{\text{expert}}^{\text{device}} = \frac{k}{EP}\left(4HI_{\text{moe}}\right)$$

and may also be TP-sharded if FFN GEMMs follow the same path:

- If experts use TP GEMMs: divide by TP again.
- If experts are **only EP-sharded**, do **not** include $1/TP$.

Dense layers always have $EP = 1$.

PP (Pipeline Parallelism)

PP assigns whole layers to stages.
Each device in a PP stage owns:

$$\frac{L}{PP}\ \text{layers.}$$

Thus:

$$F_{\text{token, device}} = \frac{L}{PP}\left(F_{\text{proj}}^{\text{device}} + F_{\text{attn,KV}}^{\text{device}} + F_{\text{ffn}}^{\text{device}} + F_{\text{norm}}\right)$$

Total Per-device FLOPs:

Droping the negligible F_{norm} and also substituting everything yields the **final fully expanded expression** per-device FLOPs for a single decoded token:

$$\boxed{F_{\text{token,device}} \approx \frac{L}{PP}\left(\frac{2H^2 + 6HH_{kv}}{TP} + \frac{4HI_{\text{eff}}}{TP \cdot EP} + \frac{4SH_{kv}}{TP \cdot SP} + 2HN_{\text{eff}}\right)}$$

For a **dense MLP model**: $I_{\text{eff}} = I_{\text{dense}}, \quad N_{\text{eff}} = 0, \quad EP = 1$

For a **MoE model**: $I_{\text{eff}} = kI_{\text{moe}}, \quad N_{\text{eff}} = N_{\text{exp}}, \quad EP \geq 1$

3.6 Prefill FLOPs (Supplementary Reference)

The expressions in Sections 3.1–3.5 apply strictly to **decoding**, where only one new token is processed per step and FLOPs scale as $O(S)$ through the $F_{\text{attn,KV}}$ term.

For completeness, we summarize the **prefill (full-sequence)** FLOPs here. Prefill is **GEMM-dominant** and substantially more expensive than decoding, but its cost is incurred **once per**

request, not per generated token.

A coarse scaling for prefill FLOPs is:

$$F_{\text{prefill}} = O(BSLH^2) \; + \; O(BLS^2 H_{kv})$$

- The first term corresponds to projections and FFN blocks across all S input tokens.
- The second term corresponds to full attention score computation over the $S \times S$ attention matrix per layer.

Why this section is supplementary:
Prefill FLOPs do **not** affect decoding throughput (TPS_{single}) and do not enter the per-device roofline model (Sections 5.5, 7). They only contribute to **TTFT** (time-to-first-token), which is modeled in Section 9.

---

4. Compute vs. Memory Bound (Roofline Model)

Sections 2 and 3 derived:

- the **per-token memory traffic** ($T^{eff}_{\text{token,device}}$), including FlashAttention effects.
- the **per-token FLOPs per device** on this PP stage ($F_{\text{token,device}}$), and

We now combine them using a standard roofline model to determine the **local per-token latency**.

Let:

- R_{GPU}: sustained device compute throughput (FLOPs/s),
- $B_{\text{eff,mem}}$: sustained device memory bandwidth (bytes/s).

Both reflect *sustained* performance, not peak specs.

4.1 Operational Intensity (Ops:Byte)

The **operational intensity** for decoding on this device is:

$$\text{OI} = \frac{F_{\text{token,device}}}{T^{eff}_{\text{token,device}}} \quad (\text{FLOPs per byte})$$

High-level interpretation:

- **High OI** → more FLOPs per byte → *compute-bound*
- **Low OI** → fewer FLOPs per byte → *memory-bound*

This rating is compared to the device's memory-to-compute ratio:

$$\frac{R_{\text{GPU}}}{B_{\text{eff,mem}}}$$

If:

$$\text{OI} > \frac{R_{\text{GPU}}}{B_{\text{eff,mem}}} \quad \Rightarrow \quad \text{compute-bound}$$

else:

$$\text{OI} < \frac{R_{\text{GPU}}}{B_{\text{eff,mem}}} \quad \Rightarrow \quad \text{memory-bound}$$

Dominant-Term Approximation

In practice, the OI is often approximated using only the largest FLOP and traffic terms:

- FLOPs dominated by:

$$\max\left(2H^2,\ 4HI_{\text{eff}},\ 4SH_{kv}/SP\right)$$

- Memory traffic dominated by the KV term:

$$\frac{2SH_{kv}}{TP \cdot SP}\, b$$

Thus for long-context decoding:

$$\text{OI} \approx \frac{4SH_{kv}/(TP \cdot SP)}{2SH_{kv}/(TP \cdot SP)\ b} = \frac{2}{b}$$

which explains why long-context dense models almost always appear **memory-bound**.

4.2 Compute-Bound Time

Given the per-token FLOPs on this device (Section 3.5):

$$t_{\text{compute}} = \frac{F_{\text{token,device}}}{R_{\text{GPU}}}$$

This is the time assuming unlimited memory bandwidth.

4.3 Memory-Bound Time

Given the effective memory traffic per token (Section 2.4):

$$t_{\text{mem}} = \frac{T^{eff}_{\text{token,device}}}{B_{\text{eff,mem}}}$$

This is the time assuming compute is free.

For long-context LLMs, this term is often dominant due to KV-cache reads.

4.4 Local Device Time Bound

The per-token latency for this PP stage is:

$$t_{\text{local}} = \max\left(t_{\text{compute}},\ t_{\text{mem}}\right)$$

This is the roofline principle:

> **Decoding is limited by whichever is slower — compute throughput or memory bandwidth.**

---

5. Communication Time During Decoding

Section 4 defined the **local per-token latency** on each device:

$$t_{\text{local}} = \max(t_{\text{compute}}, t_{\text{mem}})$$

We now incorporate the **inter-device communication time** that arises during decoding under distributed parallelism. In the nested parallelism structure used throughout this document:

$$\mathbf{PP} \;\rightarrow\; \mathbf{EP} \;\rightarrow\; \mathbf{TP} \;\rightarrow\; \mathbf{SP}$$

each axis contributes its own per-token communication term. All communication costs follow the standard α–B model:

$$t_{\text{comm}} = \alpha + \frac{\text{message size}}{B_{\text{eff}}}$$

where α is the collective or hop latency, and B_{eff} is the sustained bandwidth of the communication path.

The parameters α and B_{eff} in this model are not abstract: they are **topology-dependent physical properties** of the underlying interconnect. Different parallelism domains—TP, EP, SP, and PP—may be mapped to **different network fabrics** or different portions of the same physical topology (e.g., NVSwitch star within a node, 2D/3D torus across nodes, or hybrid switch-plus-fabric designs). Consequently, each collective type sees its own communication characteristics, with potentially different latency constants and effective bandwidths. To keep the analysis general, we denote these as α_{XP} and $B_{\text{eff},XP}$ for TP, EP, SP, and PP respectively. Their actual numerical values depend on the system's physical layout, routing scheme, and bisection bandwidth properties (e.g., constant-hop NVSwitch vs. hop-scaling torus fabrics). The following sections therefore use α_{XP} and $B_{\text{eff},XP}$ as **collective-specific, topology-aware** parameters, to be instantiated according to the actual deployment mapping.

Message sizes and their shard structure

To remain consistent with the compute and memory models, we strictly define the payload size for each collective type. Note the distinction between *storage size* (sharded) and *communication payload* (often full-width):

- **PP (Pipeline Parallel):** Uses **activation shards** of width $\approx H/TP$. *Rationale:* High-performance PP (e.g., Megatron-LM) preserves TP rank alignment, so only the local TP shard needs to be forwarded to the next stage.
- **EP (Expert Parallel):** Uses **full activations** of width $k \cdot H$. *Rationale:* MoE routing sends token activations to experts. While the traffic is bidirectional (Dispatch + Combine), we model this by applying a factor of 2 to the *collective steps* in Section 5.2 rather than doubling the base message size here.

- **TP (Tensor Parallel):** Uses **full hidden state vectors** of width H. *Rationale:* Row-Parallel matrix multiplication produces a vector of **partial sums** that has the full width H. These must be All-Reduced across ranks, requiring the transfer of the full vector, not a shard.
- **SP (Sequence Parallel):** Uses **KV-cache blocks** of size $\frac{S}{SP} \cdot \frac{2H_{kv}}{TP}$. *Rationale:* Ring Attention streams the distributed KV blocks around the ring. Each rank receives and passes chunks of this size in a continuous stream.

5.1 Pipeline Parallel (PP) Hop

Pipeline Parallelism (PP) forwards activations from one pipeline stage to the next. Because TP is nested inside PP, high-performance implementations (e.g., Megatron-LM PP, DeepSpeed PP, NVIDIA NeMo, and FasterTransformer) preserve the **TP rank alignment** across all PP stages. That is, TP rank i in stage s corresponds directly to TP rank i in stage $s+1$.

This alignment has an important consequence:
each device only needs to forward its own TP shard of the hidden state, not a full H-dimensional vector. The full activation is conceptually transferred across the PP boundary, but it is split naturally across TP separate device-to-device links.

Thus the PP hop behaves as a **single, shard-sized point-to-point transfer**, with message size $\approx H/TP$ per device.

For a single token, the latency of this hop is modeled as:

$$t_{PP} = \alpha_{PP} + \frac{(H/TP)\, b}{B_{\text{eff,PP}}}$$

This shard-preserving PP design avoids the extra TP collectives that would be required if stages exchanged full activations and then re-sharded them. Maintaining TP rank consistency across stages therefore yields a significantly faster pipeline, and is the standard strategy in modern LLM training and inference systems.

5.2 Expert Parallel (EP) All-to-All (MoE Dispatch and Combine)

MoE layers require exchanging token activations across the expert-parallel (EP) dimension. In contrast to TP collectives (which perform a reduce-then-broadcast) or PP hops (which are unidirectional), EP communication follows a **bidirectional dispatch-and-combine pattern**:

1. **Dispatch:** Token activations are routed from the source rank to the rank holding the selected expert (top-k routing).
2. **Combine:** The expert's output must be sent **back** to the source rank to be added to the residual stream.

Because the token must traverse the link twice (once to the expert, once back), the network traffic is double that of a simplex transfer.

Let k denote the number of active experts per token. Each device transmits approximately kHb bytes per token during Dispatch and receives kHb bytes during Combine.

5.2.1 Ring-Style EP All-to-All

A ring all-to-all across EP devices performs $(EP-1)$ exchange rounds. Including the factor of 2 for the round-trip nature of MoE routing, the **per-token, per-layer** latency model is:

$$t_{EP}^{\text{ring}} = 2(EP-1)\alpha_{EP} + 2(EP-1)\frac{kH\,b}{EP \cdot B_{\text{eff,EP}}}$$

Interpretation:

- The factor of **2** accounts for the full round trip (Dispatch + Combine).
- Communication depth grows linearly with $(EP-1)$.
- Each device sends the equivalent of $2kHb$ bytes total per token.

5.2.2 Log-Style (Tree/Recursive) EP All-to-All

Optimized implementations can reduce collective depth via recursive-doubling or tree-style schedules. The **per-token, per-layer** latency can be modeled as:

$$t_{EP}^{\text{tree}} \approx 2\lceil \log_2(EP) \rceil \alpha_{EP} + 2\frac{kH\,b}{B_{\text{eff,EP}}}$$

Here:

- The exchange requires only $\lceil \log_2(EP) \rceil$ rounds per direction.
- The per-device payload remains $2kHb$ (Dispatch + Combine).

For dense models ($EP = 1$), we have $t_{EP} = 0$.

5.3 Tensor Parallel (TP) Communication

TP groups compute each layer in parallel across TP devices. The most common operation is the **All-Reduce** required at the end of the MLP (Row Parallel) and Attention (Output) blocks to sum the partial results across ranks.

Critical Note on Message Size: Unlike PP (which sends a shard), the TP All-Reduce operates on the **full hidden state vector** (H). Although each device only "owns" a shard of the weights, the partial output computed by Row Parallelism is a vector of size H (containing partial sums). These must be reduced globally.

5.3.1 TP Ring All-Reduce

For a ring all-reduce, the **per-token, per-layer** latency is:

$$t_{TP}^{\text{ring}} = 2(TP-1)\alpha_{TP} + 2\frac{TP-1}{TP} \cdot \frac{H\,b}{B_{\text{eff,TP}}}$$

Interpretation:

- The factor of **2** comes from the two phases of All-Reduce: **Reduce-Scatter** + **All-Gather**.
- The payload is the **full** hidden size H, not the shard size H/TP.
- Each device sends and receives approximately $2H$ bytes per collective (for large TP).

5.3.2 TP Tree All-Reduce

For a tree algorithm:

$$t_{TP}^{\text{tree}} \approx 2\lceil \log_2(TP) \rceil \alpha_{TP} + 2\frac{H\,b}{B_{\text{eff,TP}}}$$

Tree algorithms reduce the latency term (logarithmic steps) but often utilize the same bandwidth (sending the full vector H up and down the tree).

5.4 Sequence Parallel (SP) Communication

Sequence Parallelism (SP) in inference typically refers to **Ring Attention**. Here, the KV cache is partitioned along the sequence dimension S. To compute attention for a new token:

1. The Query (Q) remains local on the device.
2. The KV blocks are **rotated** around the ring so that the local Q can attend to the full history.

This is effectively an **All-Gather** operation (streaming the distributed KV cache to every rank), not an All-Reduce.

While tree-based variants are theoretically possible, practical implementations (e.g., Megatron-LM SP, DeepSpeed-Ulysses, and fused attention kernels) consistently use ring-style schedules: KV shards are large, non-contiguous in memory, and must be processed in left-to-right sequence order, making tree-style SP impractical.

Thus, for modeling purposes, we assume **ring-style SP communication**.

SP Ring Communication Latency

A ring with SP ranks performs $(SP-1)$ uni-directional shifts. Unlike the two-pass scatter-gather structure of TP All-Reduce, Ring Attention is a **single-pass** streaming operation.

The **per-token, per-layer** latency can be expressed as:

$$t_{SP} = (SP-1)\alpha_{SP} + (SP-1) \cdot \frac{\left(\frac{S}{SP} \cdot \frac{2H_{kv}}{TP}\right) b}{B_{\text{eff,SP}}}$$

Interpretation:

- **1 Pass:** We do not scatter partial results; we simply stream the KV shards.
- **Message Size:** In each step, we pass the local KV shard. The total volume transferred per device is the size of the *entire* rest of the KV cache: $\frac{SP-1}{SP} \times \text{TotalKV}$.

5.5 Total Communication Time Per Token on a PP Stage

Sections 5.1–5.4 provide **per-token, per-layer** communication costs for each parallelism axis (TP, EP, SP), and a **per-token, per-hop** cost for PP.
We now clarify how these terms combine to form the total per-token communication time on a given pipeline-parallel (PP) stage.

Per-layer vs. per-stage normalization

A Transformer layer contains exactly one Attention block and one MLP/MoE block. Each block triggers a fixed number of communication collectives, and within each layer, TP, EP, and SP collectives are strictly ordered:

- **Attention**
  - 1 TP collective (Output Projection)
  - 1 SP collective (if SP is enabled)
- **MLP (dense)**
  - 1 TP collective (Output Projection)
- **MoE block**
  - 1 EP all-to-all
  - 1 TP collective (Expert Output Projection)

These collectives must complete before the token can advance to the next layer. Since a PP stage contains L/PP *sequential* layers, the total communication work for that stage is:

- n_{TP} TP collectives per layer
- n_{EP} EP collectives per layer (0 for dense layers, 1 for MoE layers)
- n_{SP} SP collectives per layer (1 if SP is enabled)

Because TP, EP, and SP operations within each layer depend on one another (e.g., TP → SP in attention and EP → TP in MoE), they are **strictly sequential** and do not overlap. Thus, the **per-token communication time accumulated over the entire PP stage** is:

$$t_{\text{comm,stage}} = \frac{L}{PP}\big(n_{TP}\, t_{TP} + n_{EP}\, t_{EP} + n_{SP}\, t_{SP}\big)$$

Where:

- t_{EP}, t_{TP}, t_{SP} are the **per-token, per-layer** communication costs given in Sections 5.2–5.4.
- n_{TP} is typically **2** (one for Attention, one for FFN).
- n_{EP} is **1** for MoE layers and **0** for dense layers.
- n_{SP} is **1** (occurs during Attention).

Adding PP hop cost

The PP hop is different: it is a **per-token, per-hop** cost rather than a per-layer cost. A token is forwarded once from PP stage s to stage $s{+}1$, with latency t_{PP} as defined in Section 5.1.

Thus, the total per-token communication time for this stage is:

$$\boxed{t_{\text{comm}} = \frac{L}{PP}\big(n_{TP}\, t_{TP} + n_{EP}\, t_{EP} + n_{SP}\, t_{SP}\big) + t_{PP}}$$

Interpretation

- The first term accumulates **all TP, EP, and SP collectives** required by the L/PP layers on this PP stage.
- The second term accounts for the **one PP hop** that forwards the token to the next stage.
- This combined expression represents the **total communication work** per token for the stage. Whether this communication becomes the latency bottleneck or is hidden by overlap is addressed in Section 4's roofline-style model and Section 6's end-to-end pipeline analysis.

Summary of Collective Types and Message Sizes

Parallelism	Occurs in	Collective Type	Passes	Message Size (per device)
PP	between layers	point-to-point	1	$(H/TP)\, b$
TP	attn + FFN	all-reduce (ring/tree)	2	$H\, b$
EP	MoE FFN	all-to-all	2	$kH\, b$
SP	attention	all–gather (ring)	1	$(S/SP) \cdot (2H_{kv}/TP)\, b$

Practical Guidance: When to Use Ring vs. Tree Collectives

The choice between ring-style and tree-style collective algorithms depends strongly on the **physical interconnect topology**:

- **NVSwitch / fully-connected crossbars (constant-hop latency):**
  Ring collectives often provide higher effective bandwidth because all links can operate concurrently with no hop-distance penalty.

- **2D/3D torus, dragonfly, and multi-hop switch fabrics:**
  Tree or recursive-doubling collectives typically reduce depth and latency, since ring communication cost scales with hop count. Low-latency links benefit more from tree scheduling.

- **Sequence Parallelism (SP):**
  SP operations must traverse KV shards in strict left-to-right order. Tree-style SP is rarely implemented in practice; **ring is the standard**.

This guidance explains why Section 5 provides both **ring** and **tree** expressions for EP and TP, but uses **ring** exclusively for SP.

---

6. End-to-End Latency, Throughput, and Partition Strategy

This section integrates memory limits, compute, communication, and pipeline behavior into a complete end-to-end decoding performance model. We discuss:

1. Feasible model partitioning via **HBM limits**
2. Local per-token latency via the **compute–memory roofline**
3. Per-token communication latency from **TP/EP/SP/PP**
4. Overlap strategies between compute and communication
5. Throughput (**TPS**, **TTPS**) based on PP-stage bottlenecks
6. **TTFT**, including prefill on a separate cluster and PP-stage latency summation

---

6.1 Model Partition Strategy from HBM Constraints

A parallel configuration (DP, PP, EP, TP, SP) is **feasible** only if each device can store:

- its **parameter shard**,
- its **KV-cache shard**, and
- the **activation workspace** needed for a single decoding token,

within the available HBM capacity M_{HBM}.

We define the total per-device static footprint as

$$M_{\text{device}}^{\text{total}} = M_{\theta,\text{device}} + M_{\text{act,device}} + M_{\text{KV,device}} \ \leq \ M_{\text{HBM}}$$

Using the per-device memory expressions derived in Section 1, the **fully expanded** form is:

$$\frac{L}{PP}\left[\frac{H^2 + 3HH_{kv}}{TP} \ + \ \frac{3HIN_{\text{exp}}}{TP \cdot EP} + \ (4H + 2H_{kv}) + \frac{2SH_{kv}}{TP \cdot SP}\right] b + \frac{VH}{TP} b \leq \ M_{\text{HBM}}$$

where:

- the bracketed term is the **intermediate PP-stage** footprint (parameters, activations, KV cache).
- and the final $\frac{VH}{TP} b$ term models the **worst-case embedding / LM-head overhead** on boundary PP stages.
- for a **dense MLP model**: $I = I_{\text{dense}}$, and $N_{\text{exp}} = EP = 1$.
- for a **MoE model**: $I = I_{\text{moe}}$, with $N_{\text{exp}} > 1$ and $EP \geq 1$.

Calculating DP for a Fixed Total HBM Capacity

The total Data Parallelism degree (DP) is constrained by both the total cluster size (N_{GPUs}) and the memory headroom available on each device.

1. **Memory Headroom Requirement:** DP scaling is only possible if $M_{\text{device}}^{\text{total}} \leq M_{\text{HBM}}$ for the chosen inner sharding degrees (PP, EP, TP, SP).
2. **Replication Logic:** Each model replica requires a dedicated group of $PP \cdot EP \cdot TP \cdot SP$ devices.

Let N_{GPUs} be the total number of devices in the cluster. The maximum achievable DP count is:

$$\boxed{DP = \left\lfloor \frac{N_{\text{GPUs}}}{PP \cdot EP \cdot TP \cdot SP} \right\rfloor}$$

Physical Interpretation:

- **Scaling Limit:** To increase DP for higher throughput ($TTPS$), one must either add more total GPUs to the cluster or increase inner sharding (e.g., higher PP or SP) to reduce $M_{\text{device}}^{\text{total}}$, though the latter consumes more devices per replica.
- **Footprint vs. Replica Count:** There is a direct trade-off: higher sharding degrees "thin out" the memory footprint per device to fit large context S or large models, but they simultaneously reduce the number of independent replicas that can fit in a fixed cluster.

6.2 Local and Networking Per-Token Latency

Compute-bound latency

$$t_{\text{compute}} = \frac{F_{\text{token,device}}}{R_{\text{GPU}}}$$

Memory-bandwidth-bound latency

$$t_{\text{mem}} = \frac{T_{\text{token,device}}^{\text{eff}}}{B_{\text{eff,mem}}}$$

Roofline local latency

$$t_{\text{local}} = \max(t_{\text{compute}},\ t_{\text{mem}})$$

Collective communication latency

$$t_{\text{comm}} \approx \frac{L}{PP}\left(n_{TP}\, t_{TP} + n_{EP}\, t_{EP} + t_{SP}\right) + t_{PP}$$

Unified Overlap Model

We introduce an overlap factor $\rho \in [0, 1]$ representing the fraction of local compute/memory time that is successfully utilized to hide communication.

The effective per-token latency is the local time plus any **unhidden** communication:

$$t_{\text{token}} = t_{\text{local}} + \max\left(0,\ t_{\text{comm}} - \rho \cdot t_{\text{local}}\right)$$

Regimes:

- $\rho = 0$ **(No Overlap):**

$$t_{\text{token}} = t_{\text{local}} + t_{\text{comm}}$$

  Typical for naive implementations or strictly sequential dependencies.

- $\rho = 1$ **(Perfect Overlap Opportunity):**

$$t_{\text{token}} = t_{\text{local}} + \max(0, t_{\text{comm}} - t_{\text{local}}) = \max(t_{\text{local}}, t_{\text{comm}})$$

  Achieved by highly optimized kernels (e.g., Ring Attention) where independent work exists.

- $0 < \rho < 1$ **(Partial Overlap):** Models real-world overheads (kernel launch latency, synchronization barriers) that prevent utilizing the full local duration for hiding comms.

6.3 TPS and TTPS — Pipeline Throughput

Recall from Section 6.2 that for a given PP stage we model the **per-token, per-stage** latency using the unified overlap model:

$$t_{\text{token}} = t_{\text{local}} + \max\big(0,\ t_{\text{comm}} - \rho \cdot t_{\text{local}}\big)$$

For pipeline stage j, we write this explicitly as

$$t_{\text{stage},j} = t_{\text{local},j} + \max\big(0,\ t_{\text{comm},j} - \rho \cdot t_{\text{local},j}\big)$$

During steady-state decoding, different PP stages process **different tokens** concurrently. The overall throughput of a single DP replica is therefore limited by the **slowest** pipeline stage:

$$TPS_{\text{single}} \approx \frac{1}{\max_j t_{\text{stage},j}}$$

Across DP fully independent replicas (no cross-replica coupling), the total cluster throughput scales linearly:

$$TTPS \approx DP \cdot TPS_{\text{single}} = DP \cdot \frac{1}{\max_j t_{\text{stage},j}}$$

Here:

- The **throughput bottleneck** is $\max_j t_{\text{stage},j}$ (the slowest stage).
- The **pipeline depth / first-token traversal time** is $\sum_{j=1}^{PP} t_{\text{stage},j}$, which appears in the TTFT expression in Section 6.4.

6.4 TTFT — Time To First Token (Prefill on a Separate Cluster)

This section models TTFT assuming the **prefill runs on a separate cluster**, and the decoding cluster receives the final KV cache before emitting the first token.

For clarity, TTFT consists of three sequential phases:

1. **Prefill latency** (could be on a separate cluster)
2. **KV-cache transfer**
3. **Pipeline traversal delay** (empty pipeline warm-up)

Only after all PP stages have processed the first token can the first decoded token be emitted.

Prefill latency (other cluster)

Let the prefill cluster have:

- Compute rate: $R_{\text{GPU,pre}}$
- Memory bandwidth: $B_{\text{eff,mem,pre}}$

Then:

$$t_{\text{prefill}} \approx \max\left(\frac{F_{\text{prefill}}}{R_{\text{GPU,pre}}}, \frac{T_{\text{prefill}}}{B_{\text{eff,mem,pre}}}\right)$$

KV-cache transfer

Total KV size:

$$M_{\text{KV,total}} = 2SH_{kv}b \cdot L$$

If inter-cluster link bandwidth is $B_{\text{link,cluster}}$:

$$t_{\text{KV-transfer}} \approx \frac{M_{\text{KV,total}}}{B_{\text{link,cluster}}}$$

Pipeline traversal latency (decoding cluster)

The first decoded token must traverse all PP stages **sequentially**, since the pipeline is empty:

$$t_{\text{decode,first-token}} \approx \sum_{j=1}^{PP} t_{\text{stage},j}$$

Final TTFT

$$TTFT \approx t_{\text{prefill}} + t_{\text{KV-transfer}} + \sum_{j=1}^{PP} t_{\text{stage},j} + t_{\text{startup}}$$